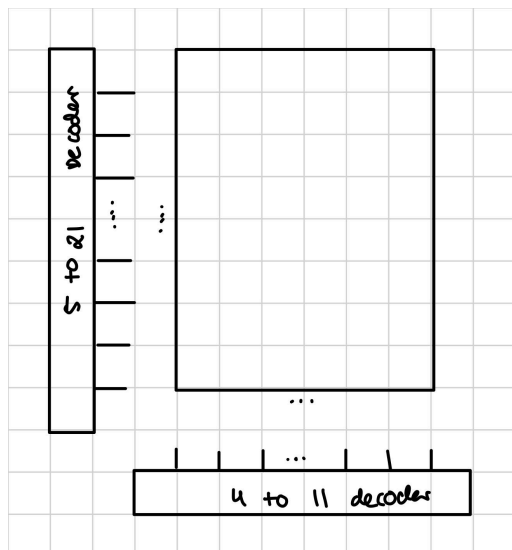


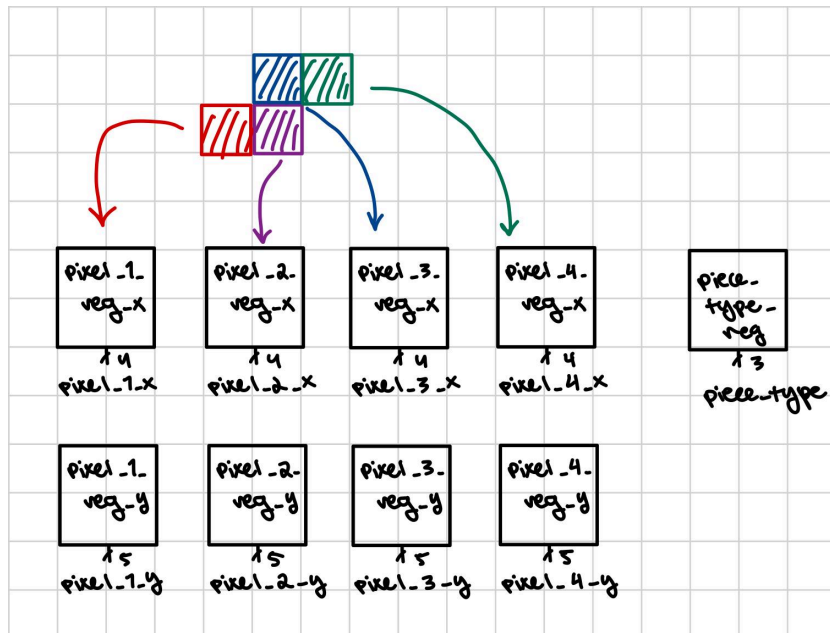
Rough design:

The main strategy of this design is to use the fact that the VGA cycle is really long to allow for game state to be updated over multiple clock cycles instead of one.

The tetris grid is roughly 20x12. To represent the pieces that are not the current one, I was thinking of using a grid of flip-flops/registers which can be selected like memory. If I'm tight on space, I might just use single-bit flip-flops to keep track of whether the space is occupied or not without worrying what particular color the space is. If I have room, it might be nice to keep track of color (or the piece type which is 3 bits). There are three signals being fed to the flip-flops/regs: read, write, and shift. Read and write perform like normal memory. Shift is used for clearing full rows, so the flip-flops will instead load whatever value the flip-flop above them has.



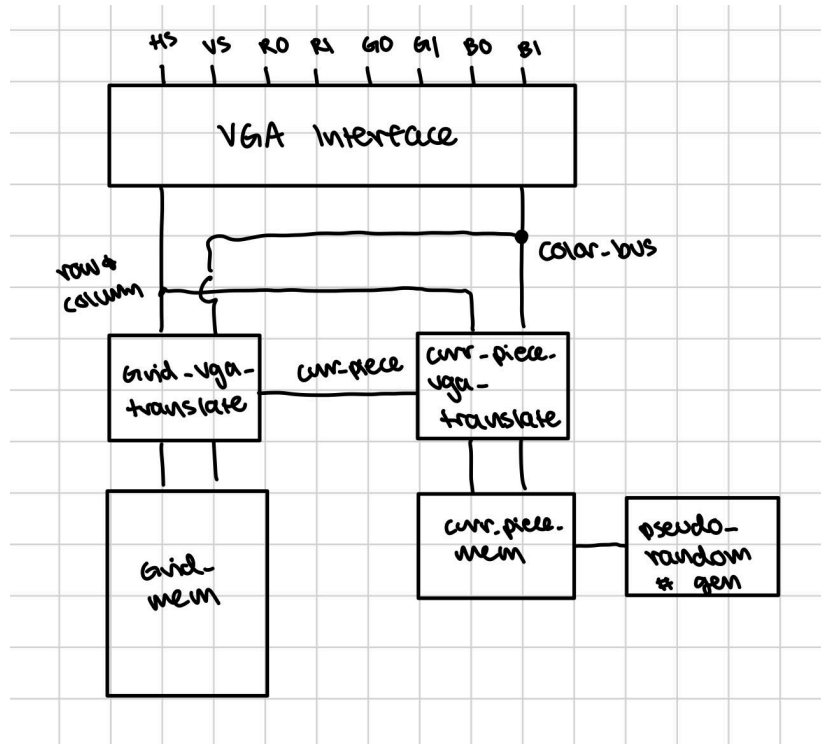
The current piece that is being controlled is always four pixels. So we can just have 8 registers to keep track of the location of each of these pixels and an additional register to keep track of the piece type.



The general flow of the design would be as follows:

- Evaluate what the next state of the piece is based on user inputs. For rotating, the arithmetic operations to find the next state can be hard coded depending on piece type.
- Take four cycles to evaluate if the next state positions are valid. This includes making sure they are in-bounds and empty by reading each of the pixel locations from the tetris grid.
- If it's valid, commit the next state to state.
- If it's not valid, either ignore the next state. Or, if the next state was generated because the piece is falling, take four cycles to load the piece to the grid.
- Repeat the above 4 steps if the user input was a drop until the piece is as low as it can be.
- If I have time, I could probably implement the shadow piece in a similar way.

I'm thinking each of the components of the screen will have their own color translator and drive a shared bus based on if the row/column bus matches their coordinates. The colors for all the components will most likely be hardcoded, so the number of bits used for the color_bus will be less than 6.



Some extra components/details:

- I was thinking of using a small pseudo-random number generator to determine what the next piece should be. If it's too big I can also just use a set pattern.
- There will be a stored piece register.
- The pieces when falling into the grid will all start in hard-coded positions.
- No fancy game play like t-spins.
- Score will be limited to 32 bits? Or however many will fit.

Parts:

VGA Pmod: [digikey](#)

- Tiny tapeout has a [tiny pmod](#) that my ports will match. But, if I can figure out how the tiny pmod works, I could maybe use the normal VGA Pmod for prototyping?
- Edit: Nace has some

VGA display: could I use one of the one's for 240

Joystick: the only ones I could find all use [i2c](#) instead of SPI?

- Edit: Nace also has some

Speaker: something like [this](#)? I know 220 has many